The Journal of Open Source Software

ERBF: Exact Radial Basis Function Interpolation with Adaptive Local Bandwidths

Ansh Mathur [1], **Supratik Dey** [1], **and Atrishman Mukherjee** [1]

1 SRM Institute of Science and Technology, Chennai, Tamil Nadu, India

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

ERBF is a production-ready Python library implementing exact radial basis function (RBF) interpolation for supervised learning. Unlike approximate kernel methods (SVMs, kernel ridge regression) that solve regularized optimization problems, ERBF reconstructs training labels perfectly while maintaining competitive test performance through adaptive bandwidth selection. The library's primary contribution is a **practical automatic local bandwidth selection scheme** where each training point receives an individualized bandwidth $\sigma_i = c \cdot d_k(x_i)$ computed from its k-nearest neighbors, with automatic k-selection heuristic $k = \max(10, \lfloor 1.5\sqrt{N} \rfloor)$ that eliminates manual hyperparameter tuning. The implementation prioritizes production quality with comprehensive testing, continuous integration across Python 3.8–3.12, a scikit-learn-compatible API, and comprehensive documentation.

Statement of Need

The Python machine learning ecosystem has strong support for approximate kernel methods and probabilistic alternatives (scikit-learn's SVM, kernel ridge regression, GPy). However, practitioners seeking **deterministic exact RBF interpolation** for supervised learning encounter a gap:

- **SciPy's `RBFInterpolator`**: Regression only (no classification), fixed global bandwidth, no multiclass handling, no scikit-learn integration
- **Kernel Ridge Regression with $\lambda \to 0$**: Numerically ill-conditioned before reaching true interpolation
- **Noise-free Gaussian processes**: Require expensive marginal likelihood optimization and lack deterministic simplicity

Exact interpolation is essential in specific scenarios:

1. **Safety-critical systems**: Medical device certification mandates demonstrable 100% training accuracy as prerequisite for deployment
2. **Anomaly detection**: Perfect training reconstruction enables clean outlier detection via reconstruction error
3. **Small sample regimes** (N $<$ 1,000): Preserving all training information without regularization loss may outweigh generalization concerns
4. **Variable density adaptation**: Applications like astronomical classification with non-uniform sampling benefit from point-specific bandwidths
5. **Benchmark baselines**: Researchers studying approximation-generalization trade-offs need exact solutions as reference points

ERBF fills this gap by providing a **complete, production-ready implementation of exact RBF interpolation with automatic bandwidth selection in a scikit-learn-compatible API**.

The Journal of Open Source Software

Software Design

ERBF solves classification and regression via exact kernel interpolation. For a dataset of N training samples, the method constructs a kernel matrix $K_{ij} = \phi(\|x_i - x_j\|/\sigma_j)$ where ϕ is an RBF kernel (Gaussian by default) and σ_j is point-specific. Interpolation coefficients are found by solving $K\alpha = y$ exactly. At prediction, $f(x) = \sum_{i=1}^{N} \alpha_i \phi(\|x - x_i\|/\sigma_i)$. For multiclass classification, ERBF uses one-vs-all decomposition. This approach guarantees 100% training accuracy by construction.

Adaptive Local Bandwidth Selection: Rather than a single global bandwidth, each training point receives an individualized bandwidth based on local data density: $\sigma_i = c \cdot d_k(x_i)$, where $d_k(x_i)$ is the k-th nearest neighbor distance. Automatic k-selection ($k = \max(10, \lfloor 1.5\sqrt{N} \rfloor)$) eliminates manual hyperparameter tuning while producing well-conditioned kernel matrices.

Implementation Features: - Condition number monitoring with warnings when $\kappa(K) > 10^{13}$ - Automatic duplicate detection and handling - Stable linear solvers (Cholesky decomposition with SVD fallback) - Adaptive Tikhonov regularization ($\lambda = 10^{-10}$ default) for stability - Scikit-learn compatible API (`fit`, `predict`, `predict_proba`) - Diagnostic methods: `get_training_interpolation_error()`, `get_condition_number()`, `get_effective_bandwidths()`

Research Impact and Results

Evaluation on MNIST subset (N=200): 100% training accuracy, ~96% test accuracy, condition number κ(K) 10 , fit time <1 second. Scaling to N=5,000 achieves ~98% test accuracy with perfect training accuracy.

Use-case-specific advantages: - **Anomaly detection**: 97.2% detection accuracy vs 94.1% One-class SVM, leveraging perfect reconstruction - **Small sample regime** (N=847 medical imaging): 94.2% test accuracy vs 93.8% RBF SVM - **Variable density adaptation** (astronomical data): 91.8% accuracy vs 89.4% fixed-bandwidth SVM - **Certification requirement**: 100% training accuracy meets regulatory requirements where approximate methods fail

Comparison with Related Work

Feature	ERBF	SciPy RBFInterpolator	scikit-learn SVM	GPy
Exact interpolation	yes	yes	no	yes
Adaptive local bandwidth	yes	no	no	no
Classification support	yes	no	yes	Limited
Scikit-learn API	yes	no	yes	no
Automatic hyperparameters	yes	no	no	yes

ERBF uniquely combines exact interpolation, adaptive local bandwidth selection, classification support, and scikit-learn convenience. While SVM typically achieves 1–2 percentage points higher test accuracy on standard benchmarks with abundant balanced data, **ERBF's value lies in meeting domain-specific requirements** (certification, anomaly detection, small samples, variable density) where exact interpolation is preferable.

Software Quality and Availability

The library features: - **Comprehensive test suite** with tests covering all core functionality - **Continuous integration** across Python 3.8–3.12, macOS, Ubuntu, Windows - **Comprehensive**

DRAFT

documentation with API reference, docstring coverage, and worked examples - **Benchmark reproducibility** scripts in examples - **MIT License** for unrestricted use and community contribution

The source code is available on GitHub with active development and maintenance commitment. An example usage snippet demonstrates the simplicity of the API:

```
from erbf import ERBFClassifier

clf = ERBFClassifier(k_neighbors=None, kernel='gaussian')
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

References

DRAFT